DAY 27

Day 27: Backpressure, Flow Control & Load Shedding

Keeping systems stable when incoming traffic exceeds processing capacity.

1 Backpressure Basics

- Producers can outpace consumers
- Unbounded queues increase latency
- Backpressure keeps the system stable

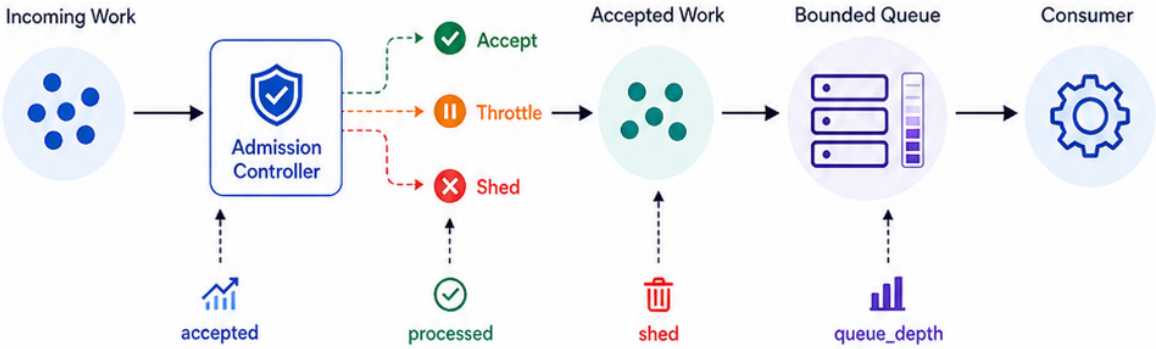
2 Bounded Queues & Lag

- Queue capacity is a safety boundary
- Queue depth signals overload
- Lag shows consumers are falling behind

3 Throttling & Load Shedding

- Throttling slows intake near saturation
- Shedding rejects excess or low-priority work
- Protects core throughput under bursts

4 Admission Control Flow



5 Operational Behavior

- Metrics guide policy tuning
- Accepted / processed / shed are key signals
- Policies should be deterministic and observable

6 Trade-Offs

- More shedding → more stability, more dropped work
- Less shedding → higher latency and timeout risk
- Tune policy to workload criticality and SLAs

Graceful overload handling protects throughput and prevents system collapse.

System Design - Day 27

Backpressure, Flow Control, and Load Shedding

Prepared from Day-27 notes and sample implementation

Overview from README.md

Day 27: Backpressure, Flow Control, and Load Shedding

Day 27 focuses on how systems stay stable when incoming traffic exceeds processing capacity.

It covers:
- backpressure fundamentals
- bounded queue flow control
- lag and saturation signals
- intake throttling strategies
- load shedding under overload
- reliability and latency trade-offs

In short, Day 27 is about protecting the system from collapse by controlling admission when demand spikes.

Core ideas

Why backpressure matters
- Producers can generate work faster than consumers can process it.
- Without control, queues grow unbounded and latency explodes.
- Backpressure keeps the system within safe operating limits.

Bounded queues and lag
- A queue capacity cap is a hard safety boundary.
- Queue depth and lag are primary indicators of overload.
- Once depth crosses thresholds, the system should reduce intake or shed load.

Throttling and shedding
- `Throttling` slows admission to match downstream capacity.
- `Load shedding` rejects lower-priority or excess work to preserve core throughput.
- These are resilience tools, not failure states.

Operational behavior
- Overload handling should be explicit and measurable.
- Metrics like `accepted`, `processed`, `shed`, and `queue_depth` guide tuning.
- Policies should be deterministic to simplify incident analysis.

Trade-offs
- Aggressive shedding improves stability but drops more work.
- Lenient admission reduces drops but risks long latency and timeout cascades.
- Policy tuning depends on business SLAs and workload criticality.

Day-27 sample: Backpressure Admission Controller

This repository includes a small backpressure simulation in both Python and Java.

Functional requirements
- Accept incoming work into a bounded queue
- Process work at fixed consumer throughput per tick
- Apply watermark-based admission policy
- Throttle intake near saturation

- Shed load at high queue pressure
- Report operational metrics and lag trend

High-level components
- `WorkItem`: unit of queued work
- `BoundedQueue`: capacity-limited in-memory queue
- `AdmissionController`: watermark policy for accept/throttle/shed
- `Consumer`: fixed-rate processor
- `BackpressureEngine`: tick-based simulation and metrics

Data flow
1. Incoming batch arrives for a simulation tick
2. Admission controller decides per item (accept/throttle/shed)
3. Accepted items enter bounded queue
4. Consumer drains up to configured throughput
5. Metrics and queue depth are recorded for tuning analysis

Project structure

```
Day-27/
README.md
backpressure-sample/
python/
backpressure_controller.py
demo.py
java/
BackpressureSystem.java
Main.java
```

Run the sample

Python

```
cd backpressure-sample\python
python demo.py
```

Java

```
cd backpressure-sample\java
javac Main.java BackpressureSystem.java
java Main
```

Interview takeaways
- Backpressure prevents overload from becoming a full outage
- Queue depth is both a control input and an SLO risk signal
- Throttling and shedding should be policy-driven and observable
- Stability often requires rejecting some work under bursty load
- Reliability design includes graceful degradation, not just retries

Next improvements
- Add request priority classes for selective shedding
- Add adaptive consumer scaling policy
- Add exponential backoff hints to producers
- Add end-to-end latency percentile tracking
- Add alert thresholds for sustained high queue depth

Reference Implementation

Python and Java sample source files

File: backpressure-sample/python/demo.py

```python
from backpressure_controller import BackpressureEngine


def main() -> None:
    engine = BackpressureEngine(
        queue_capacity=10,
        consumer_rate_per_tick=3,
        throttle_watermark=0.6,
        shed_watermark=0.9,
    )

    incoming_batches = [
        ["m1", "m2", "m3", "m4"],
        ["m5", "m6", "m7", "m8", "m9", "m10", "m11"],
        ["m12", "m13", "m14", "m15", "m16", "m17"],
        ["m18", "m19", "m20"],
        ["m21", "m22", "m23", "m24", "m25"],
    ]

    print("tick results")
    for tick_index, batch in enumerate(incoming_batches, start=1):
        result = engine.tick(batch)
        print(f"  tick={tick_index} in={len(batch)} -> {result}")

    print("\nsummary")
    print(f"  accepted={engine.accepted}")
    print(f"  throttled={engine.throttled}")
    print(f"  shed={engine.shed}")
    print(f"  processed={engine.processed}")
    print(f"  final_queue_depth={engine.queue.depth()}")
    print(f"  depth_history={engine.depth_history}")


if __name__ == "__main__":
    main()
```

File: backpressure-sample/python/backpressure_controller.py

```python
from __future__ import annotations

from collections import deque
from dataclasses import dataclass


@dataclass(frozen=True)
class WorkItem:
    id: str


class BoundedQueue:
    def __init__(self, capacity: int) -> None:
        if capacity <= 0:
            raise ValueError("capacity must be positive")
        self._capacity = capacity
        self._items: deque[WorkItem] = deque()

    def enqueue(self, item: WorkItem) -> bool:
        if len(self._items) >= self._capacity:
            return False
        self._items.append(item)
        return True

    def dequeue_many(self, count: int) -> list[WorkItem]:
        result: list[WorkItem] = []
        while count > 0 and self._items:
            result.append(self._items.popleft())
            count -= 1
        return result

    def depth(self) -> int:
        return len(self._items)

    def capacity(self) -> int:
        return self._capacity


class AdmissionController:
    def __init__(self, throttle_watermark: float, shed_watermark: float) -> None:
        if not (0 < throttle_watermark < shed_watermark < 1):
            raise ValueError("watermarks must satisfy 0 < throttle < shed < 1")
        self._throttle_watermark = throttle_watermark
        self._shed_watermark = shed_watermark

    def decide(self, queue: BoundedQueue, item_index: int) -> str:
        fill_ratio = queue.depth() / queue.capacity()

        if fill_ratio >= self._shed_watermark:
            return "SHED"
        if fill_ratio >= self._throttle_watermark:
            # Deterministic throttling: accept every 2nd item under pressure.
            return "ACCEPT" if item_index % 2 == 0 else "THROTTLE"
        return "ACCEPT"


class BackpressureEngine:
    def __init__(
        self,
        queue_capacity: int,
        consumer_rate_per_tick: int,
        throttle_watermark: float,
        shed_watermark: float,
    ) -> None:
        if consumer_rate_per_tick <= 0:
            raise ValueError("consumer_rate_per_tick must be positive")
        self.queue = BoundedQueue(queue_capacity)
        self.controller = AdmissionController(throttle_watermark, shed_watermark)
        self.consumer_rate = consumer_rate_per_tick

        self.accepted = 0
        self.throttled = 0
```

```
        self.shed = 0
        self.processed = 0
        self.depth_history: list[int] = []

    def tick(self, incoming_ids: list[str]) -> dict[str, int]:
        accepted_now = 0
        throttled_now = 0
        shed_now = 0

        for index, item_id in enumerate(incoming_ids):
            decision = self.controller.decide(self.queue, index)
            item = WorkItem(id=item_id)

            if decision == "ACCEPT":
                if self.queue.enqueue(item):
                    accepted_now += 1
                else:
                    shed_now += 1
            elif decision == "THROTTLE":
                throttled_now += 1
            else:
                shed_now += 1

        processed_now = len(self.queue.dequeue_many(self.consumer_rate))

        self.accepted += accepted_now
        self.throttled += throttled_now
        self.shed += shed_now
        self.processed += processed_now
        self.depth_history.append(self.queue.depth())

        return {
            "accepted_now": accepted_now,
            "throttled_now": throttled_now,
            "shed_now": shed_now,
            "processed_now": processed_now,
            "queue_depth": self.queue.depth(),
        }
```

File: backpressure-sample/java/Main.java

```java
import java.util.List;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        BackpressureSystem.BackpressureEngine engine = new BackpressureSystem.BackpressureEngine(
            10,
            3,
            0.6,
            0.9
        );

        List<List<String>> incomingBatches = List.of(
            List.of("m1", "m2", "m3", "m4"),
            List.of("m5", "m6", "m7", "m8", "m9", "m10", "m11"),
            List.of("m12", "m13", "m14", "m15", "m16", "m17"),
            List.of("m18", "m19", "m20"),
            List.of("m21", "m22", "m23", "m24", "m25")
        );

        System.out.println("tick results");
        int tick = 1;
        for (List<String> batch : incomingBatches) {
            Map<String, Integer> result = engine.tick(batch);
            System.out.println("  tick=" + tick + " in=" + batch.size() + " -> " + result);
            tick += 1;
        }

        System.out.println("\nsummary");
        System.out.println("  accepted=" + engine.getAccepted());
        System.out.println("  throttled=" + engine.getThrottled());
        System.out.println("  shed=" + engine.getShed());
        System.out.println("  processed=" + engine.getProcessed());
        System.out.println("  final_queue_depth=" + engine.getQueueDepth());
        System.out.println("  depth_history=" + engine.getDepthHistory());
    }
}
```

File: backpressure-sample/java/BackpressureSystem.java

```java
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Queue;

public class BackpressureSystem {
    public static final class WorkItem {
        private final String id;

        public WorkItem(String id) {
            this.id = id;
        }

        public String getId() {
            return id;
        }
    }

    public static final class BoundedQueue {
        private final int capacity;
        private final Queue<WorkItem> items = new ArrayDeque<>();

        public BoundedQueue(int capacity) {
            if (capacity <= 0) {
                throw new IllegalArgumentException("capacity must be positive");
            }
            this.capacity = capacity;
        }

        public boolean enqueue(WorkItem item) {
            if (items.size() >= capacity) {
                return false;
            }
            items.add(item);
            return true;
        }

        public List<WorkItem> dequeueMany(int count) {
            List<WorkItem> result = new ArrayList<>();
            while (count > 0 && !items.isEmpty()) {
                result.add(items.remove());
                count -= 1;
            }
            return result;
        }

        public int depth() {
            return items.size();
        }

        public int capacity() {
            return capacity;
        }
    }

    public static final class AdmissionController {
        private final double throttleWatermark;
        private final double shedWatermark;

        public AdmissionController(double throttleWatermark, double shedWatermark) {
            if (!(0 < throttleWatermark && throttleWatermark < shedWatermark && shedWatermark < 1)) {
                throw new IllegalArgumentException("watermarks must satisfy 0 < throttle < shed < 1");
            }
            this.throttleWatermark = throttleWatermark;
            this.shedWatermark = shedWatermark;
        }

        public String decide(BoundedQueue queue, int itemIndex) {
            double fillRatio = (double) queue.depth() / queue.capacity();
            if (fillRatio >= shedWatermark) {
```

```
            return "SHED";
        }
        if (fillRatio >= throttleWatermark) {
            return itemIndex % 2 == 0 ? "ACCEPT" : "THROTTLE";
        }
        return "ACCEPT";
    }
}

public static final class BackpressureEngine {
    private final BoundedQueue queue;
    private final AdmissionController controller;
    private final int consumerRate;

    private int accepted = 0;
    private int throttled = 0;
    private int shed = 0;
    private int processed = 0;
    private final List<Integer> depthHistory = new ArrayList<>();

    public BackpressureEngine(
        int queueCapacity,
        int consumerRatePerTick,
        double throttleWatermark,
        double shedWatermark
    ) {
        if (consumerRatePerTick <= 0) {
            throw new IllegalArgumentException("consumerRatePerTick must be positive");
        }
        this.queue = new BoundedQueue(queueCapacity);
        this.controller = new AdmissionController(throttleWatermark, shedWatermark);
        this.consumerRate = consumerRatePerTick;
    }

    public Map<String, Integer> tick(List<String> incomingIds) {
        int acceptedNow = 0;
        int throttledNow = 0;
        int shedNow = 0;

        for (int i = 0; i < incomingIds.size(); i += 1) {
            String decision = controller.decide(queue, i);
            WorkItem item = new WorkItem(incomingIds.get(i));
            if ("ACCEPT".equals(decision)) {
                if (queue.enqueue(item)) {
                    acceptedNow += 1;
                } else {
                    shedNow += 1;
                }
            } else if ("THROTTLE".equals(decision)) {
                throttledNow += 1;
            } else {
                shedNow += 1;
            }
        }

        int processedNow = queue.dequeueMany(consumerRate).size();

        accepted += acceptedNow;
        throttled += throttledNow;
        shed += shedNow;
        processed += processedNow;
        depthHistory.add(queue.depth());

        Map<String, Integer> result = new LinkedHashMap<>();
        result.put("accepted_now", acceptedNow);
        result.put("throttled_now", throttledNow);
        result.put("shed_now", shedNow);
        result.put("processed_now", processedNow);
        result.put("queue_depth", queue.depth());
        return result;
    }

    public int getAccepted() {
```

```
            return accepted;
        }

        public int getThrottled() {
            return throttled;
        }

        public int getShed() {
            return shed;
        }

        public int getProcessed() {
            return processed;
        }

        public int getQueueDepth() {
            return queue.depth();
        }

        public List<Integer> getDepthHistory() {
            return depthHistory;
        }
    }
}
```